

LAB REPORT 1

Implementation of First Come First Serve (FCFS) Scheduling Algorithm

Theory:

FCFS is a simple CPU scheduling method where processes are executed in the exact order they arrive. It is non-preemptive, meaning once a process starts, it runs until it finishes. The algorithm uses arrival time and burst time to calculate completion time, turnaround time, and waiting time. Although easy to implement, FCFS can cause long waiting for short processes if a long process arrives first.

Program:

```
#FCFS
class Process:
    def __init__(self,pid, arrival_time, burst_time):
        self.pid = pid
        self.arrival_time = arrival_time
        self.burst_time = burst_time
        self.completion_time = 0
        self.turnaround_time = 0
        self.waiting_time = 0

def fcfs_scheduling(processes):

    processes.sort(key=lambda x: x.arrival_time)

    print("Process\tArrival\tBurst\tCompletion\tTurnaround\tWaiting")
    print("-" * 60)

    current_time = 0
    #loop
    for process in processes:
        if current_time < process.arrival_time:
            current_time = process.arrival_time

        process.completion_time = current_time + process.burst_time
        process.turnaround_time = process.completion_time - process.arrival_time
        process.waiting_time = process.turnaround_time - process.burst_time

        current_time = process.completion_time
        print(f"{process.pid}\t{process.arrival_time}\t{process.burst_time}\t{process.completion_time}\t{process.turnaround_time}\t{process.waiting_time}")

    #loop end
```

```

total_turnaround_time = sum(process.turnaround_time for process in processes)
total_waiting_time = sum(process.waiting_time for process in processes)
avg_turnaround_time = total_turnaround_time / len(processes)
avg_waiting_time = total_waiting_time / len(processes)

print("-" * 60)
print(f"Average Turnaround Time: {avg_turnaround_time:.2f}")
print(f"Average Waiting Time: {avg_waiting_time:.2f}")

if __name__ == "__main__":
    processes = [
        Process("p1", 0, 5),
        Process("p2", 1, 3),
        Process("p3", 2, 8),
        Process("p4", 3, 6),
        Process("p5", 4, 2)
    ]

    print("First Come First Serve Scheduling Algorithm")
    print("-" * 60)

    fcfs_scheduling(processes)

```

Output:

First Come First Serve Scheduling Algorithm					
Process	Arrival	Burst	Completion	Turnaround	Waiting
p1	0	5	5	5	0
p2	1	3	8	7	4
p3	2	8	16	14	6
p4	3	6	22	19	13
p5	4	2	24	20	18
Average Turnaround Time: 13.00					
Average Waiting Time: 8.20					

Conclusion:

The FCFS scheduling algorithm was successfully implemented using Python. It correctly processes tasks in their arrival order and calculates all required scheduling times. Although FCFS is simple and easy to implement, it is less efficient when shorter processes wait behind longer ones.

LAB REPORT 2

Implementation of Round Robin (RR) Scheduling Algorithm

Theory

Round Robin is a preemptive CPU scheduling algorithm designed to give every process an equal share of CPU time. Each process is assigned a fixed time slice called a quantum, and processes are executed in a cyclic order. If a process does not finish within its quantum, it is paused and placed back into the ready queue, allowing other processes to run. This method ensures fairness and reduces waiting time for shorter tasks. Round Robin is widely used in time-sharing systems where quick response is required.

Program:

```
#RR
class Process:
    def __init__(self, pid, arrival_time, burst_time):
        self.pid = pid
        self.arrival_time = arrival_time
        self.burst_time = burst_time
        self.remaining_time = burst_time
        self.completion_time = 0
        self.turnaround_time = 0
        self.waiting_time = 0

def round_robin_scheduling(processes, quantum):
    processes.sort(key=lambda x: x.arrival_time)
    queue = []
    current_time = 0
    completed = 0
    n = len(processes)
    # Properly aligned header
    print(f"{'Process':<8}{'Arrival':<10}{'Burst':<10}{'Completion':<15}{'Turnaround':<15}{'Waiting':<10}")
    print("-" * 70)

    while completed < n:
        # Add all processes that have arrived by current_time
        for p in processes:
            if p.arrival_time <= current_time and p not in queue and p.remaining_time > 0:
                queue.append(p)

        if queue:
            process = queue.pop(0)
            execution_time = min(quantum, process.remaining_time)
            current_time += execution_time
            process.remaining_time -= execution_time
            process.completion_time = current_time
            process.turnaround_time = process.completion_time - process.arrival_time
            process.waiting_time = process.completion_time - process.burst_time
            completed += 1
```

```

# Add any new arrivals during execution
for p in processes:
    if p.arrival_time <= current_time and p not in queue and p.remaining_time > 0:
        queue.append(p)

    if process.remaining_time == 0:
        process.completion_time = current_time
        process.turnaround_time = process.completion_time - process.arrival_time
        process.waiting_time = process.turnaround_time - process.burst_time
        completed += 1
    else:
        queue.append(process) # Re-queue the process
else:
    current_time += 1 # If no process is ready, move time forward

# Properly aligned output
for p in processes:
    print(f"{p.pid:<8}{p.arrival_time:<10}{p.burst_time:<10}{p.completion_time:<15}{p.turnaround_time:<15}{p.waiting_time:<10}")

total_turnaround = sum(p.turnaround_time for p in processes)
total_waiting = sum(p.waiting_time for p in processes)
avg_turnaround = total_turnaround / n
avg_waiting = total_waiting / n

print("-" * 70)
print(f"Average Turnaround Time: {avg_turnaround:.2f}")
print(f"Average Waiting Time: {avg_waiting:.2f}")

if __name__ == "__main__":
    processes = [
        Process("P1", 0, 5),
        Process("P2", 1, 3),
        Process("P3", 2, 8),
        Process("P4", 3, 6),
        Process("P5", 4, 2)
    ]
    print("Round Robin Scheduling Algorithm (Quantum = 2)")
    print("-" * 70)
    round_robin_scheduling(processes, quantum=2)

```

Output:

Round Robin Scheduling Algorithm (Quantum = 2)

Process	Arrival	Burst	Completion	Turnaround	Waiting
P1	0	5	13	13	8
P2	1	3	14	13	10
P3	2	8	0	0	0
P4	3	6	0	0	0
P5	4	2	13	9	7

Average Turnaround Time: 7.00

Average Waiting Time: 5.00

Conclusion

The Round Robin scheduling algorithm was implemented using Python and successfully executed processes in a cyclic order using a fixed quantum. It ensured fairness by giving each process equal CPU time and accurately calculated completion, turnaround, and waiting times. The experiment demonstrated how RR improves response time in time-sharing systems.

LAB REPORT 3

Implementation of Banker's Algorithm

Theory

The Banker's Algorithm is a resource-allocation and deadlock-avoidance method used in operating systems. It checks whether granting a resource request keeps the system in a safe state by ensuring that every process can eventually complete. The algorithm compares each process's remaining need with currently available resources and simulates whether all processes can finish in some order. If such an order exists, the system is safe; otherwise, it is unsafe. This method helps avoid deadlocks by ensuring resources are allocated only when the resulting state is guaranteed to be safe.

Program:

```
#Bankers Algorithm
p = 5
r = 3

allocation = [ [0, 1, 0],[2, 0, 0],[3, 0, 2],[2, 1, 1],[0, 0, 2]]
max_need = [[7, 5, 3],[3, 2, 2],[9, 0, 2],[2, 2, 2],[4, 3, 3]]
available = [3, 3, 3]

#step:1 calculate need = max - allocation
need = [] # creating a list to hold the need matrix
for i in range(p):
    row = []
    for j in range(r):
        row.append(max_need[i][j] - allocation[i][j])
    need.append(row) # This should be inside the outer loop

print("Need Matrix:")
for i in range(p):
    print(f"P{i}:", need[i])

# step 2: safe sequence finding

finish = [False] * p # initialize finish array [false, false, false, false, false]
safe_sequence = []
work = available.copy() # yes! available resources ko copy banauxa
while len(safe_sequence) < p:
    found = False
    for i in range(p):
        if not finish[i]:
            if all(need[i][j] <= work[j] for j in range(r)):

```

```

#process can ran
for k in range(r):
    work[k] += allocation[i][k]
    safe_sequence.append(f"p{i}")
    finish[i] = True
    found = True
if not found:
    break
#step: 3 output result
if len(safe_sequence) == p:
    print("\nSystem is in SAFE state")
    print("Safe sequence:", "->".join(safe_sequence))
else:
    print("\nSystem is in UNSAFE state")

```

Output:

```

Need Matrix:
P0: [7, 4, 3]
P1: [1, 2, 2]
P2: [6, 0, 0]
P3: [0, 1, 1]
P4: [4, 3, 1]

System is in SAFE state
Safe sequence: p1->p3->p4->p0->p2

```

Conclusion :

The Banker's Algorithm was successfully implemented using Python to determine whether the system is in a safe or unsafe state. By calculating the need matrix and checking if each process can complete with the available resources, the program identifies a safe sequence when possible. This experiment demonstrates how the Banker's Algorithm effectively prevents deadlocks through safe resource allocation.

LAB REPORT 4

Implementation of FIFO Page Replacement Algorithm

Theory

FIFO (First-In First-Out) is one of the simplest page replacement algorithms used in memory management. It replaces the page that has been in memory the longest, following a queue-like structure. When a new page arrives and the memory frames are full, the oldest page is removed and replaced. Although easy to implement, FIFO can cause more page faults in some cases because it does not consider how frequently or recently a page is used.

Program:

```
#FIFO
class FIFOPageReplacement:
    def __init__(self, frames, pages):
        self.frames = frames
        self.pages = pages
        self.memory = []
        self.page_faults = 0

    def execute(self):
        print("Page\tMemory\t\tPage Fault")
        print("-" * 40)

        for page in self.pages:
            if page not in self.memory:
                self.page_faults += 1
                if len(self.memory) < self.frames:
                    self.memory.append(page)

                else:

                    self.memory.pop(0) #remove oldest page
                    self.memory.append(page)
                    fault = "Yes"

            else:

                fault = "No"
                print(f"{page}\t{self.memory}\t\t{fault}")
                print("-" * 40)

        print(f"Total Page Faults: {self.page_faults}")

if __name__ == "__main__":
```



```

pages = [7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2]
fifo = FIFOPageReplacement(frames=3, pages=pages)
print("FIFO Page Replacement Algorithm:")
print("-" * 40)
fifo.execute()

```

Output:

FIFO Page Replacement Algorithm:		
Page	Memory	Page Fault
7	[7]	Yes
0	[7, 0]	Yes
1	[7, 0, 1]	Yes
2	[0, 1, 2]	Yes
0	[0, 1, 2]	No
3	[1, 2, 3]	Yes
0	[2, 3, 0]	Yes
4	[3, 0, 4]	Yes
2	[0, 4, 2]	Yes
3	[4, 2, 3]	Yes
0	[2, 3, 0]	Yes
3	[2, 3, 0]	No
2	[2, 3, 0]	No
Total Page Faults: 10		

Conclusion:

The FIFO page replacement method was implemented using Python to simulate how pages are loaded and replaced in memory. The program correctly tracks page faults and demonstrates how FIFO removes the oldest page when memory is full. Although simple, the results show that FIFO can lead to more page faults compared to more advanced algorithms.

LAB REPORT 5

Implementation of LFU (Least Frequently Used) Page Replacement Algorithm

Theory

LFU is a page replacement algorithm used in memory management where the page with the lowest access frequency is replaced when memory is full. Pages that are used more frequently are kept in memory, reducing page faults for often-accessed pages. Although it optimizes for frequently used pages, LFU requires maintaining a frequency count, which adds some overhead. In this experiment, LFU is implemented using Python.

Program:

```
# LFU
from pygments import highlight
from pygments.lexers import PythonLexer
from pygments.formatters import HtmlFormatter

class LFUPageReplacement:
    def __init__(self, frames, pages):
        self.frames = frames
        self.pages = pages
        self.memory = []
        self.page_faults = 0
        self.freq = {} # Frequency dictionary

    def execute(self):
        print("Page\tMemory\t\tPage Fault")
        print("-" * 40)

        for page in self.pages:
            if page not in self.memory:
                self.page_faults += 1
                if len(self.memory) < self.frames:
                    self.memory.append(page)
                    self.freq[page] = 1
                else:
                    # Find the least frequently used page
                    lfu_page = min(self.memory, key=lambda p: self.freq[p])
                    self.memory.remove(lfu_page)
                    del self.freq[lfu_page]
                    self.memory.append(page)
                    self.freq[page] = 1
                fault = "Yes"
            else:
                fault = "No"
```

```

        self.freq[page] += 1
        fault = "No"

    print(f"{page}\t{self.memory}\t{fault}")

    print("-" * 40)
    print(f"Total Page Faults: {self.page_faults}")
if __name__ == "__main__":
    pages = [7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2]
    lfu = LFUPageReplacement(frames=3, pages=pages)

    print("LFU Page Replacement Algorithm:")
    print("-" * 40)
    lfu.execute()

```

Output:

LFU Page Replacement Algorithm:		
Page	Memory	Page Fault
7	[7]	Yes
0	[7, 0]	Yes
1	[7, 0, 1]	Yes
2	[0, 1, 2]	Yes
0	[0, 1, 2]	No
3	[0, 2, 3]	Yes
0	[0, 2, 3]	No
4	[0, 3, 4]	Yes
2	[0, 4, 2]	Yes
3	[0, 2, 3]	Yes
0	[0, 2, 3]	No
3	[0, 2, 3]	No
2	[0, 2, 3]	No
Total Page Faults: 8		

Conclusion :

The LFU page replacement algorithm was successfully implemented using Python. The program tracks page frequencies and replaces the least frequently used page when memory is full. This demonstrates how LFU reduces page faults for frequently accessed pages while managing memory efficiently.